Search Techniques

Search is finding the records that answer a request, then putting the best ones first. This note covers fifteen techniques, from the lookup that every application does to the ones mostly found in large platforms and AI systems.

Search is not only a database feature. The same techniques run in search engines, AI applications, file systems, APIs and distributed systems: a file manager searches metadata, an API takes filter parameters, and a search engine runs one query across many machines.

How Elasticsearch indexes and scores text is in the Elasticsearch document. Embeddings, vector databases and RAG are in the LLM Applications document. This note refers to both and does not explain those parts again.

1. How the techniques are ranked

No published study measures these fifteen techniques side by side. The order below is a judgement from three checks: how many kinds of application need the technique, whether PostgreSQL, Elasticsearch, MongoDB and Redis provide it built in, and the published figures in the second table.

Rank	Technique	Where it is used	In AI and RAG systems
1	Exact-match	Every application	Steady
2	Filtered	Every application with a list screen	Required on every retrieval
3	Keyword	Every search box	Growing again, as half of hybrid
4	Full-text	Products, documents, logs	Growing again, as half of hybrid
5	Faceted	Shops, catalogs, job boards	Steady
6	Fuzzy	Search boxes, name matching	Steady
7	Metadata	Files, media, logs	Growing
8	Geospatial	Maps, delivery, travel, property	Steady
9	Vector	Recommendations, similarity, RAG	Core
10	Semantic	Help centers, documents, RAG	Core
11	Hybrid	Search engines and RAG	Fast-growing default
12	Personalized	Large shops and media platforms	Growing
13	Federated	Enterprise search, assistants	Growing
14	Graph	Social networks, fraud, knowledge graphs	Growing from a small base
15	Multimodal	Image search, visual shopping	Growing

Neighbors in this list could swap places. The tiers are firmer than the numbers. The first four are in almost every application, and the last four are mostly in large platforms or AI products.

Two different questions give two different orders. General use is the rank. The last column is growth inside AI and RAG systems, where vector, semantic and hybrid search are the techniques everything else is built around.

The published figures behind the AI column and a few of the ranks:

Technique	Published evidence
Vector	Vector databases grew 377% in a year, the fastest of any LLM-related category on the Databricks platform (State of Data + AI, 2024)
Hybrid	Built into MongoDB as `$rankFusion` (generally available June 2026) and into Redis as `FT.HYBRID` (Redis 8.4). Elasticsearch has an RRF retriever
Hybrid	Adding BM25 keyword search to embeddings cut failed retrievals by 49% (Anthropic, Contextual Retrieval, 2024)
Hybrid	Hybrid queries with semantic reranking return the most relevant results in Microsoft's benchmarks (Azure AI Search documentation)
Faceted	34% of 344 benchmarked e-commerce sites have poor filtering (Baymard Institute), which shows filtering is a standard that sites are judged on
Multimodal	More than 20 billion visual searches a month on Google Lens (Google, 2025): very large in one product, rare in ordinary applications

The vendor figures are directional. Databricks, MongoDB, Redis and Microsoft all sell search. Their numbers show where products are going, not independent market share.

2. Exact-match search

Exact-match search returns only records whose value equals the query value, character for character.

```
CREATE UNIQUE INDEX customers_email ON customers (lower(email));

SELECT id, name FROM customers WHERE lower(email) = lower(?);
```

In Spring Data the same lookup is one method, backed by that index:

```
Optional<Customer> findByEmailIgnoreCase(String email);
```

Aspect	Detail
Used for	Login by email, an order by id, a product by SKU, an invoice by number
Strength	Fast through a B-tree or hash index, and the answer is never in doubt
Limitation	One wrong character finds nothing. "ORD-9012" does not match "ord 9012"

Normalize before comparing. Lowercase emails, trim spaces, and strip separators from phone numbers, on write and on read. Most exact-match bugs are two spellings of the same value.

In a search engine, exact match needs an unanalyzed field. Why a `term` query on a `text` field finds nothing is in the Elasticsearch document, section 16.

3. Filtered search

Filtered search limits results to records that meet fixed conditions, such as a status, a price range or a date.

```
SELECT id, total, created_at
FROM orders
WHERE customer_id = ? AND status = 'SHIPPED'
  AND created_at >= DATE '2026-10-01'
ORDER BY created_at DESC
LIMIT 20;
```

Aspect	Detail
Used for	Admin screens, order history, "price under 200", tenant and permission limits
Strength	Predictable, and cheap when the filter columns are indexed
Limitation	The user must already know the values. Several filters together often return nothing

A filter is a condition, not a preference. It includes or excludes; it never changes which result is better. In Elasticsearch that difference is the filter context, covered in section 17 of the Elasticsearch document.

In RAG, the filter carries the access rules. Whether to filter before or during the vector search is covered in the LLM Applications document, section 26.

4. Keyword search

Keyword search returns records that contain the words of the query as the user typed them. It matches words, not meaning.

It is the default behind most search boxes. Small applications implement it with `LIKE`; larger ones use a search engine and search several fields at once, giving some fields more weight:

```
{
  "query": {
    "multi_match": {
      "query": "wireless headphones",
      "fields": ["name^3", "brand^2", "description"]
    }
  }
}
```

A match in `name` counts three times as much as a match in `description`, so a product named for the query ranks above one that only mentions it.

Aspect	Detail
Used for	Site search, admin search, log search, product codes and error codes
Strength	Predictable and easy to explain: the word is there, or it is not
Limitation	Vocabulary mismatch. A search for "laptop" misses a page that says "notebook"

In RAG systems, "keyword search" means BM25. It is the lexical half of hybrid search, and it finds the identifiers and rare terms that vector search misses.

5. Full-text search

Full-text search indexes all the words in long text, reduces them to a standard form, and ranks matching documents by relevance.

Keyword search says what is matched. Full-text search is how it is done well at scale: analysis, an inverted index and a relevance score. Those three parts are covered in the Elasticsearch document, sections 4 and 15.

PostgreSQL has it built in, so a separate engine is not always needed:

```
ALTER TABLE articles ADD COLUMN search tsvector
  GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED;
CREATE INDEX articles_search ON articles USING GIN (search);

SELECT id, title, ts_rank(search, q) AS rank
FROM articles, websearch_to_tsquery('english', 'refund policy') q
WHERE search @@ q
ORDER BY rank DESC
LIMIT 10;
```

Aspect	Detail
Used for	Help centers, product catalogs, documentation, email, logs
Strength	Handles plurals and word forms, and ranks the results
Limitation	Still lexical. It does not know that "cancel" and "terminate" mean the same

BM25 is the standard relevance score. Elasticsearch, OpenSearch and Solr all use it by default. PostgreSQL's `ts_rank` is simpler, which is why extensions such as ParadeDB add BM25 to Postgres.

Use Postgres until search quality needs an engine. Built-in full-text search usually covers a few million rows well. Move to Elasticsearch or OpenSearch for typo tolerance, facets and tunable ranking at scale.

6. Faceted search

Faceted search shows, beside the results, the values of several fields with a count for each, and lets the user narrow the results by clicking them.

```
Results for "running shoes"      Brand          Price
                                  Asics (41)     Under 100 (63)
                                  Nike (38)      100 to 200 (52)
                                  Brooks (17)    Over 200 (9)
```

The counts come from aggregations, run on the same query as the results:

```
{
  "query": { "match": { "name": "running shoes" } },
  "aggs": {
    "brands": { "terms": { "field": "brand" } },
    "prices": {
      "range": {
        "field": "price",
        "ranges": [ { "to": 100 }, { "from": 100, "to": 200 }, { "from": 200 } ]
      }
    }
  }
}
```

Aspect	Detail
Used for	Online shops, job boards, property sites, library catalogs
Strength	The user sees what exists before choosing, so a filter never leads to zero results
Limitation	Every query also computes the counts, which costs time on large result sets

A facet is a filter that shows its counts. It is the filtered search of section 3, with the possible values listed for the user.

Selected facets need care. When a user picks Asics, the Brand facet should still list the other brands with their counts, or the user cannot change their mind. Elasticsearch does this with `post_filter`, which filters the results after the aggregations run.

7. Fuzzy search

Fuzzy search matches words that are close to the query word, so a typing mistake still finds the result.

Closeness is usually edit distance: the number of single-letter insertions, deletions or replacements needed to turn one word into the other. "headphnes" is one edit from "headphones".

```
{ "match": { "name": { "query": "headphnes", "fuzziness": "AUTO" } } }
```

`AUTO` allows no edits for words up to two letters, one edit up to five letters, and two edits above that. In PostgreSQL the `pg_trgm` extension does a similar job by comparing three-letter pieces of words:

```
SELECT name FROM customers
WHERE name % 'Jonh Smith'
ORDER BY similarity(name, 'Jonh Smith') DESC;
```

Aspect	Detail
Used for	Search boxes on phones, people's names, addresses, company names
Strength	Recovers results that a typing mistake would lose
Limitation	Slower than exact matching, and it can match the wrong word: "cat" is one edit from "car"

Short words are where fuzzy search goes wrong. That is why `AUTO` allows no edits on very short words. Product codes and ids should never be fuzzy; a near match on an order number is a wrong answer.

8. Metadata search

Metadata search finds objects by the facts recorded about them, such as owner, type, date, tags or camera model, rather than by their content.

It is how we search things whose content cannot be searched directly: images, videos, audio, binary files and log events.

```
CREATE INDEX files_metadata ON files USING GIN (metadata);

SELECT path FROM files
WHERE metadata @> '{"project": "launch", "type": "image"}'
  AND created_at > now() - interval '30 days';
```

Aspect	Detail
Used for	File storage, media libraries, document management, logs and traces by tag
Strength	Works for any kind of object, and is fast and exact
Limitation	Only as good as the metadata. A file that nobody tagged cannot be found

Metadata search is filtered search over descriptive fields. The difference is that it is the main search, not a step that narrows another one. Cloud storage now treats it as a feature: Amazon S3 can keep object metadata in tables that are queried with SQL.

Generate metadata instead of asking people for it. File type, size, dates, the camera's EXIF data and the uploading user are recorded automatically. Tags that depend on a person are the ones that end up missing.

9. Geospatial search

Geospatial search finds records by location: within a distance of a point, inside an area, or nearest first.

A spatial index groups nearby points so a query reads only the area it needs. Two common structures:

Index	How it works
Geohash	Encodes a point as a string. Nearby points usually share a prefix
R-tree	Nests rectangles around groups of shapes. PostGIS uses it, through GiST

With PostGIS, the restaurants within 5 km, nearest first:

```
SELECT name
FROM restaurants
WHERE ST_DWithin(location, ST_MakePoint(77.59, 12.97)::geography, 5000)
ORDER BY location <-> ST_MakePoint(77.59, 12.97)::geography
LIMIT 20;
```

The same search in Elasticsearch is a `geo_distance` filter on a `geo_point` field. The Redis commands are in the Redis document, section 7.

Aspect	Detail
Used for	Delivery, ride hailing, store finders, property search, travel
Strength	Fast distance and area queries over millions of points
Limitation	Straight-line distance is not travel time. A shop across a river can be close and an hour away

Longitude comes first. PostGIS, GeoJSON and Elasticsearch arrays take longitude, then latitude. Reversing them moves Bangalore to Svalbard in the Arctic, and no error is reported.

10. Vector search

Vector search finds the stored vectors closest to a query vector. Anything that can be turned into a vector, such as text, an image, a product or a user, can be searched this way.

How vectors are indexed, and which store to choose, is covered in the LLM Applications document, sections 24 to 27. In PostgreSQL with `pgvector`, the search itself is ordinary SQL:

```
SELECT id, title
FROM products
ORDER BY embedding <=> ?   -- cosine distance to the query vector
LIMIT 10;
```

Aspect	Detail
Used for	Recommendations, similar products, duplicate detection, RAG retrieval
Strength	Finds similarity that no keyword or rule can describe
Limitation	Results are hard to explain, and it is weak on exact identifiers

Vector search is the mechanism, not the goal. Semantic search in the next section is one use of it. "More like this" product suggestions, finding duplicate support tickets and matching a photo to a catalog are others.

11. Semantic search

Semantic search finds results by meaning. The LLM Applications document, section 28, defines it and compares it with keyword search.

Most semantic search runs in two stages, and the second one is the part that is often missing:

```
query → vector search, top 100   (fast, approximate)
      → rerank, top 10           (slow, precise)
```

A reranker reads the query and each candidate together and scores how well that candidate answers the query. That is too slow for millions of documents and accurate enough to fix the order of a hundred.

Aspect	Detail
Used for	Help centers, internal documents, support answers, RAG
Strength	Finds the answer when the user's words differ from the document's
Limitation	Needs an embedding model, and the cost of re-embedding everything when the model changes

Reranking is often the cheapest improvement. It changes no index and no embeddings, and it can be added in front of any first-stage search, keyword or vector.

12. Hybrid search

Hybrid search runs keyword search and vector search on the same query and merges the two result lists into one.

The usual merge is Reciprocal Rank Fusion (RRF). It ignores the raw scores, which are on different scales, and uses only each document's position in each list:

```
score(d) = Σ  1 / (k + rank of d in that list)        k is usually 60
```

A document that is third in the keyword list and fifth in the vector list scores 1/63 + 1/65. A document that is first in one list and missing from the other scores 1/61. Appearing high in both wins.

```
{
  "retriever": {
    "rrf": {
      "retrievers": [
        { "standard": {
            "query": { "match": { "body": "refund for order 9012" } } } },
        { "knn": { "field": "body_vector", "query_vector": [0.12, -0.03],
                   "k": 50, "num_candidates": 200 } }
      ]
    }
  }
}
```

Aspect	Detail
Used for	RAG retrieval, site search, support search
Strength	Finds both "order 9012" and "my money has not come back" in one query
Limitation	Two indexes to build and keep in step, and two searches per query

Score fusion is the alternative to rank fusion. It normalizes both scores to the range 0 to 1 and adds them with weights. It can be tuned more finely, and it needs an evaluation set to choose the weights, while RRF works without tuning.

Hybrid search with a reranker is the usual RAG setup in 2026. Keyword and vector search each find candidates the other misses, and the reranker orders the merged list.

13. Personalized search

Personalized search ranks the same query differently for different users, using what we know about each user.

```
{
  "query": {
    "function_score": {
      "query": { "match": { "name": "running shoes" } },
      "functions": [
        { "filter": { "terms": { "brand": ["Asics", "Brooks"] } }, "weight": 2 }
      ],
      "boost_mode": "multiply"
    }
  }
}
```

The brands come from the user's history. The relevance score still decides what matches; the user's preferences only change the order.

Aspect	Detail
Used for	Large shops, streaming services, social feeds, job sites
Strength	Puts what this user is likely to want first
Limitation	No history for new users, privacy rules, and results that keep showing the same kind of item

Boost, never filter. Personalization that removes results hides everything outside the user's past, and the user cannot see what is missing. Raising preferred results keeps the rest reachable.

It needs data that most applications do not have. Useful personalization needs a history of searches, clicks and purchases for each user, which only platforms with many users collect.

14. Federated search

Federated search sends one query to several separate systems at the same time and combines their results into one list.

```
import java.util.Arrays;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;

public class FederatedSearch {

    record Hit(String source, String title) {}

    record Source(String name, long delayMillis, String... titles) {
        List<Hit> search(String query) {
            try {
                Thread.sleep(delayMillis);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            return Arrays.stream(titles).map(t -> new Hit(name, t)).toList();
        }
    }
```

Each source is called at the same time, and a slow source is cut off after one second:

```
    public static void main(String[] args) {
        List<Source> sources = List.of(
                new Source("wiki", 100, "Refund policy", "Refund approval steps"),
                new Source("tickets", 200, "Refund failed for order 9012"),
                new Source("drive", 3000, "Refunds Q3.xlsx"));

        List<CompletableFuture<List<Hit>>> calls = sources.stream()
                .map(s -> CompletableFuture.supplyAsync(() -> s.search("refund"))
                        .completeOnTimeout(List.of(), 1, TimeUnit.SECONDS))
                .toList();

        calls.stream()
                .flatMap(call -> call.join().stream())
                .map(hit -> hit.source() + ": " + hit.title())
                .forEach(System.out::println);
    }
}
```

Output:

```
wiki: Refund policy
wiki: Refund approval steps
tickets: Refund failed for order 9012
```

Aspect	Detail
Used for	Enterprise search, AI assistants connected to many tools, library catalogs
Strength	No copy of the data. Each system keeps its own data and its own permissions
Limitation	As slow as the slowest source, and the scores from different systems cannot be compared

Merge by rank, not by score. Each source scores on its own scale, so the RRF method from section 12 is the usual way to combine the lists.

The alternative is one central index. Copying everything into one search engine gives faster and better-ranked results, and costs a pipeline per source plus permission rules copied from every system. Federation avoids both, which is why AI assistants that connect to many tools use it.

15. Graph search

Graph search finds records by following the relationships between them, rather than by matching their fields.

In Neo4j's Cypher language, people that a user may know, ranked by mutual friends:

```
MATCH (me:Person {id: 41})-[:KNOWS]->(friend)-[:KNOWS]->(other:Person)
WHERE other <> me AND NOT (me)-[:KNOWS]->(other)
RETURN other.name, count(friend) AS mutual
ORDER BY mutual DESC
LIMIT 10;
```

Aspect	Detail
Used for	"People you may know", fraud rings, supply chains, access paths, knowledge graphs
Strength	Questions about connections several steps away stay fast
Limitation	Needs a graph model of the data, and a separate store in most systems

The question shape decides it. "Which accounts share a device with an account that was flagged for fraud?" is three joins in SQL and one short pattern in a graph. A question about one record's fields gains nothing from a graph.

GraphRAG brings it into AI systems. Entities and their relationships are extracted from documents into a graph, so a question that needs facts from several documents can follow the links instead of hoping the right chunks land near each other.

16. Multimodal search

Multimodal search lets a query in one form, such as text, an image or audio, find results in another form.

It works because a multimodal embedding model maps text and images into the same vector space, so a sentence and a photo of the same thing get similar vectors:

```
text query   →  text encoder   →  vector  →  nearest catalog image vectors
photo query  →  image encoder  →  vector  →  nearest catalog image vectors
```

Aspect	Detail
Used for	Shopping by photo, finding a scene in video, searching image libraries with words
Strength	Searches content that has no useful text
Limitation	Larger vectors and more compute, and quality depends heavily on the model

It is vector search with a different encoder. The index, the distance and the store are the same as in section 10. What changes is the model that produces the vectors, and the model must be the same one for the query and the catalog.

The pieces together

Technique	In one line
Exact-match	Equal value, through an index, after normalizing
Filtered	Fixed conditions that include or exclude
Keyword	The query's words, as typed
Full-text	Keyword search with analysis and BM25 ranking
Faceted	Filters that show counts for each value
Fuzzy	Close words, by edit distance
Metadata	Objects found by the facts recorded about them
Geospatial	Distance and area, through a spatial index
Vector	Nearest vectors, for anything that can be embedded
Semantic	Meaning, usually with a reranker as the second stage
Hybrid	Keyword and vector lists merged with RRF
Personalized	The same query, ordered for this user
Federated	One query sent to many systems, merged by rank
Graph	Relationships followed several steps away
Multimodal	Text and images in one vector space

For example, an online sports shop. A customer types "runing shoes": fuzzy search corrects the typo, full-text search ranks the products, and facets show brand and price with counts. A returning customer who usually buys Asics sees Asics first, because personalization raises them. The order page finds an order by its exact number, and the warehouse screen filters orders by status and date.

The shop adds a photo search, which is multimodal search over the product images. The store finder is a geospatial query for shops within 10 km. The support assistant uses hybrid search with a reranker over help articles and past tickets, filtered to the customer's own orders.

Most search problems are solved by choosing the right technique for the question, not by tuning one technique harder. The first question to ask is whether the user knows the exact value, the words, or only the meaning.